ARTIFICIAL INTELLIGENCE
lecture 5

Outlines

1. **Adversarial Search**

Adversarial search

- Multi-agent environments:
  - Cooperative.
  - Competitive.
- In competitive environments, different agents have conflicting goals.
- This gives rise to adversarial search, also known as **games**.

Adversarial search

- In games, other agents:
  - Want our agent to lose.
  - Introduce uncertainty, don't know what will they do.
- The unpredictability of other agents can introduce many possible **contingencies.**
- We will consider a specific type:
  - Deterministic.
  - Fully observable.
  - Two agents with alternating actions.

Adversarial search

- The main problem with games is that the search space is too huge:
  - A search tree for chess has an average branching factor of 35.
  - An average chess game lasts for 50 moves per player.
- The average search tree has 35^{100} nodes.
- There is no enough time to search the whole tree and calculate the exact consequences of every move.
- If the optimal decision is not feasible (due to time or memory constraints), some decision has to be taken.

Adversarial search

We will investigate:

- What is an optimal move and how to find it.
- Pruning the search tree.
- How to choose a good move when time is limited.
- How to define a heuristic evaluation function to approximate the utility of a state..

Adversarial search

Minimax Algorithm:

- First thing to consider is how to make an optimal move.
- Certain assumptions exist:
  - A two player game.
  - The play is sequential (agents taking turns).
  - The opponent is playing perfectly.
- The two players are referred to as MAX and MIN:
  - MAX wants to maximize the utility.
  - MIN wants to minimize the utility.

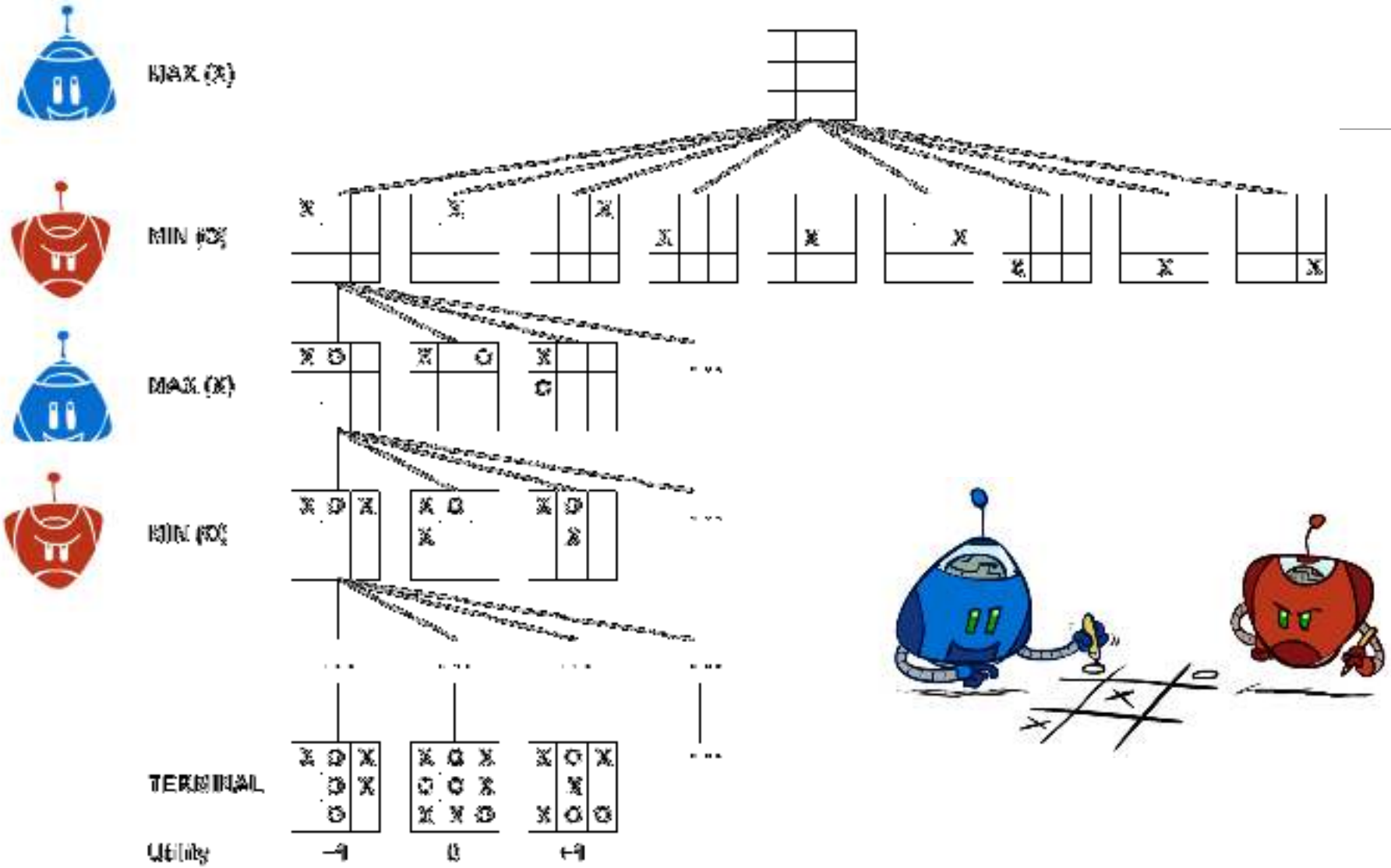
Minimax Algorithm:

A **game can be formally defined as a kind of search** problem with the following elements :

- **S0: initial state; how game is set up at start.**
- **PLAYER(s):** which player has the move in a state.
- **ACTIONS(s):** set of legal moves in a state.
- **RESULT(s; a): transition model; result of a move.**
- **TERMINAL-TEST(s): terminal test; true if and only if the** game is over. States where game ends are called **terminal states.**
- **UTILITY(s): utility function (a.k.a., payoff function or** objective function); final numeric value for a game that ends in a terminal state s.

S0, ACTIONS, and RESULT define the **game tree.**

Minimax Algorithm:



Minimax Algorithm

- **MAX is the first player to move, starting at the root** node.
- The utility values at the leaf nodes are from **MAX's** point of view.
- The optimal strategy is found by examining the **minimax value of each node.**

Minimax Algorithm

- In a normal search tree, a solution is to find a path from initial state to goal state (i.e., a wining state for **MAX).**
- However, since **MIN affects the search, a contingent** strategy should be found.
- The strategy defines:
  - **MAX's move at the initial state.**
  - **MAX's move at all states generated by all possible** moves by **MIN.**
  - ... and so on.
- This means that **MAX needs a winning strategy** independent of what **MIN does.**

Minimax Algorithm

- **MAX will choose a move that maximizes its utility value.**
- **MIN will choose a move that minimizes the utility value** of **MAX.**
- This results in **MAX (at the root) choosing its best move** given available information.
- Available information is via look ahead (the search down the tree).

Minimax Algorithm

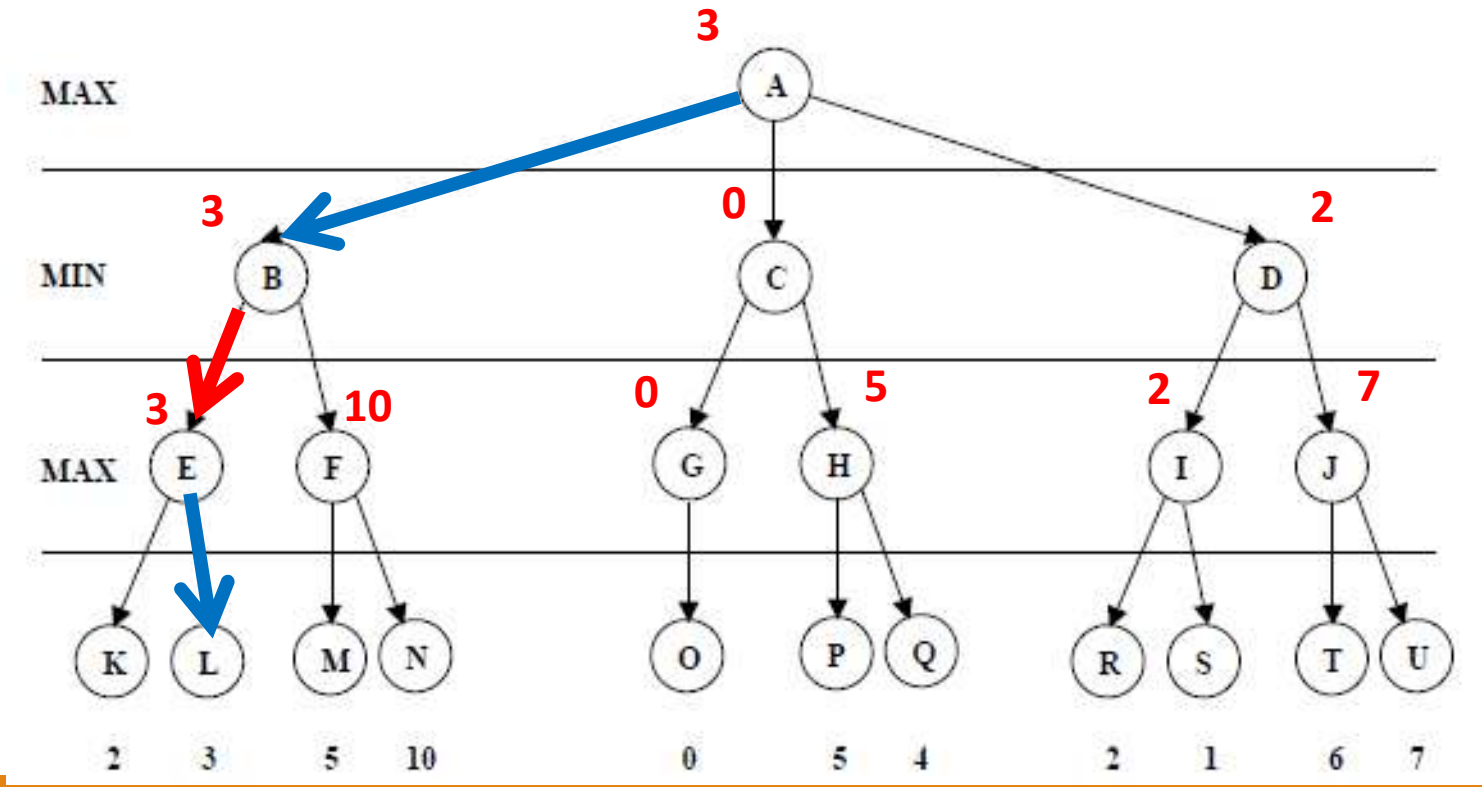
The minimax value of a node is defined as follows:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$

Minimax Algorithm

Example: applying minimax

Calculate the minimax value for the node A.



Minimax Algorithm

The minimax search algorithm

```
function MINIMAX-SEARCH(s) returns an action
    v ← MAX-VALUE(s)
    return the action in ACTIONS(s) with value v

function MAX-VALUE(s) returns a utility value
    if TERMINAL-TEST(s) then return UTILITY(s)
    v ← −∞
    for each a in ACTIONS(s) do
        v ← MAX(v,MIN-VALUE(RESULT(s,a)))
    return v

function MIN-VALUE(s) returns a utility value
    if TERMINAL-TEST(s) then return UTILITY(s)
    v ← +∞
    for each a in ACTIONS(s) do
        v ← MIN(v,MAX-VALUE(RESULT(s,a)))
    return v
```

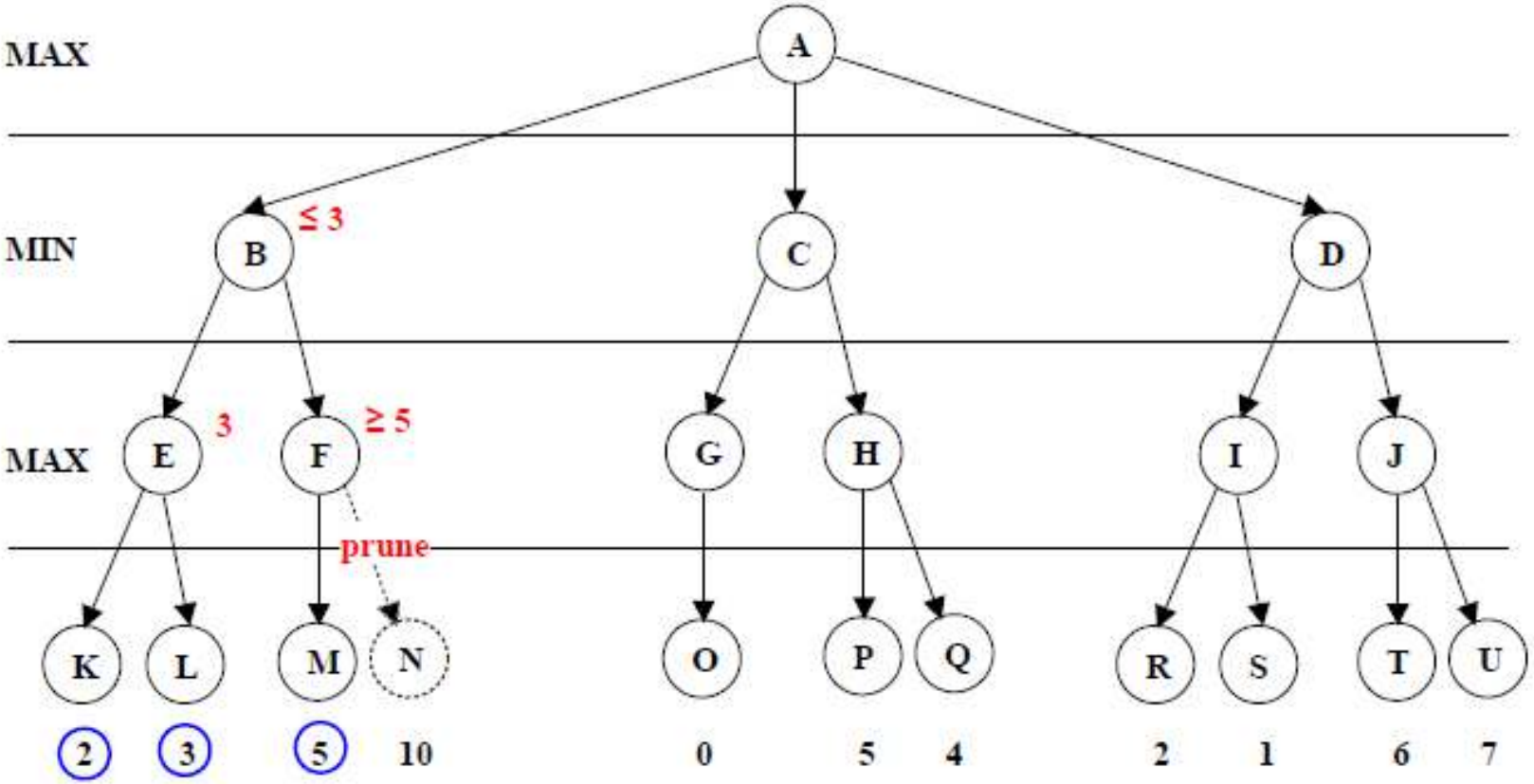
Properties of the minimax search algorithm

- Complete if the tree is finite.
- Optimal against an optimal opponent.
  - No other strategy could do any better in this case.
  - For non-optimal opponents, outcome cannot be worse
  - but there could be another strategy with better outcome!
- Time complexity = $O(b^m)$.
- Space complexity = O(bm).

β –α Pruning

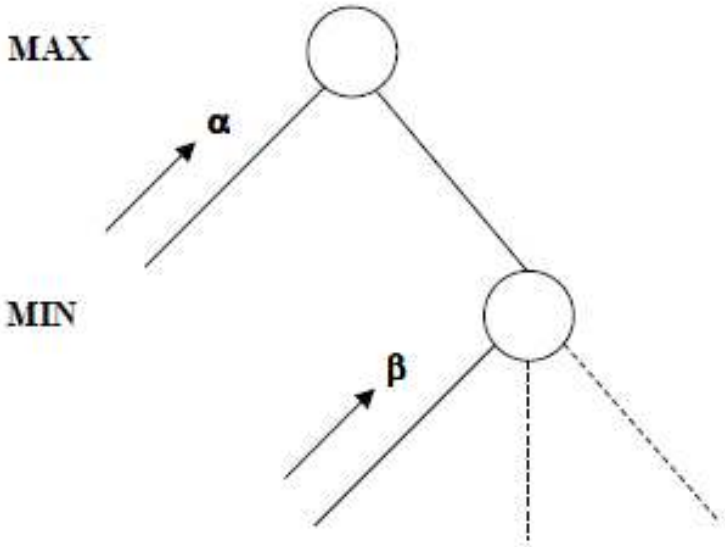
- Game trees are too huge to be solved to optimality.
- Minimax has to generate the entire tree, compute the utility values at the leaf nodes, and then propagate these values up the tree.
- We want a way to do less work by removing unnecessary branches thus exploring less nodes.
- However, we want to guarantee the same decision as Minimax.
- We use the alpha-beta (α- β) pruning technique.

Example of β –α Pruning



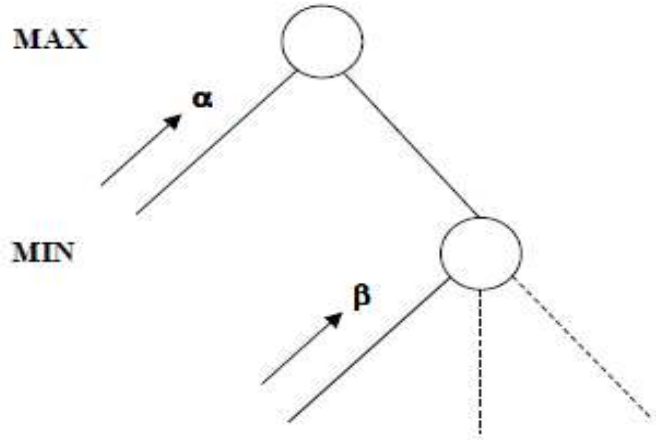
β –α Pruning

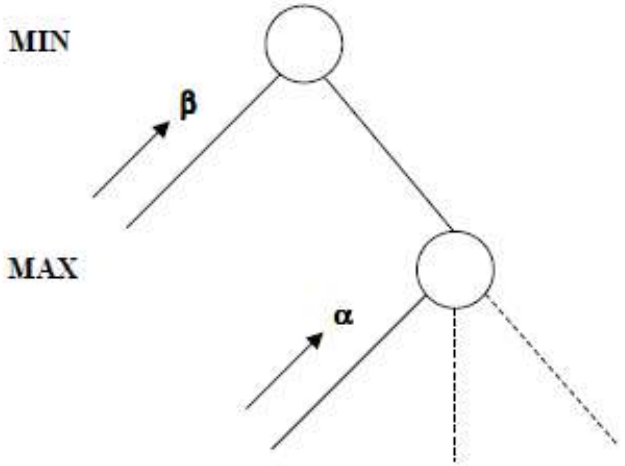
- α is the best (maximum) value **MAX is assured so** far off the current path.
- If is worse than (from **MAX's point of view),**
- **MAX will avoid it, dashed** branches will be pruned.
- Happens when α>=β
- referred to as α-cutoff.



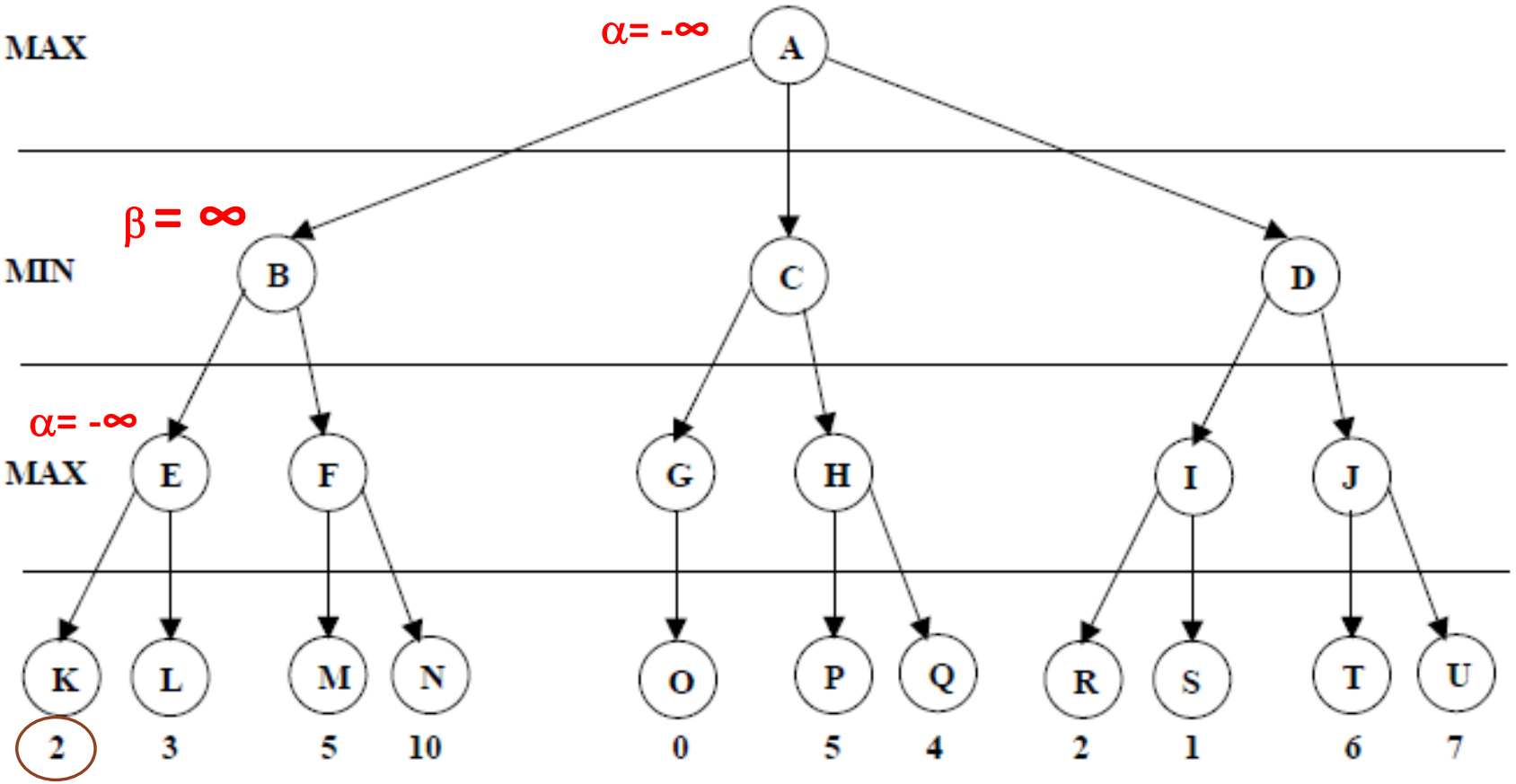
β –α Pruning

- β is the best (minimum) value **MIN is assured so** far off the current path.
- If is worse than (from **MIN's point of view),**
- **MIN will avoid it, dashed** branches will be pruned.
- Happens when α>=β
- referred to as β-cutoff.

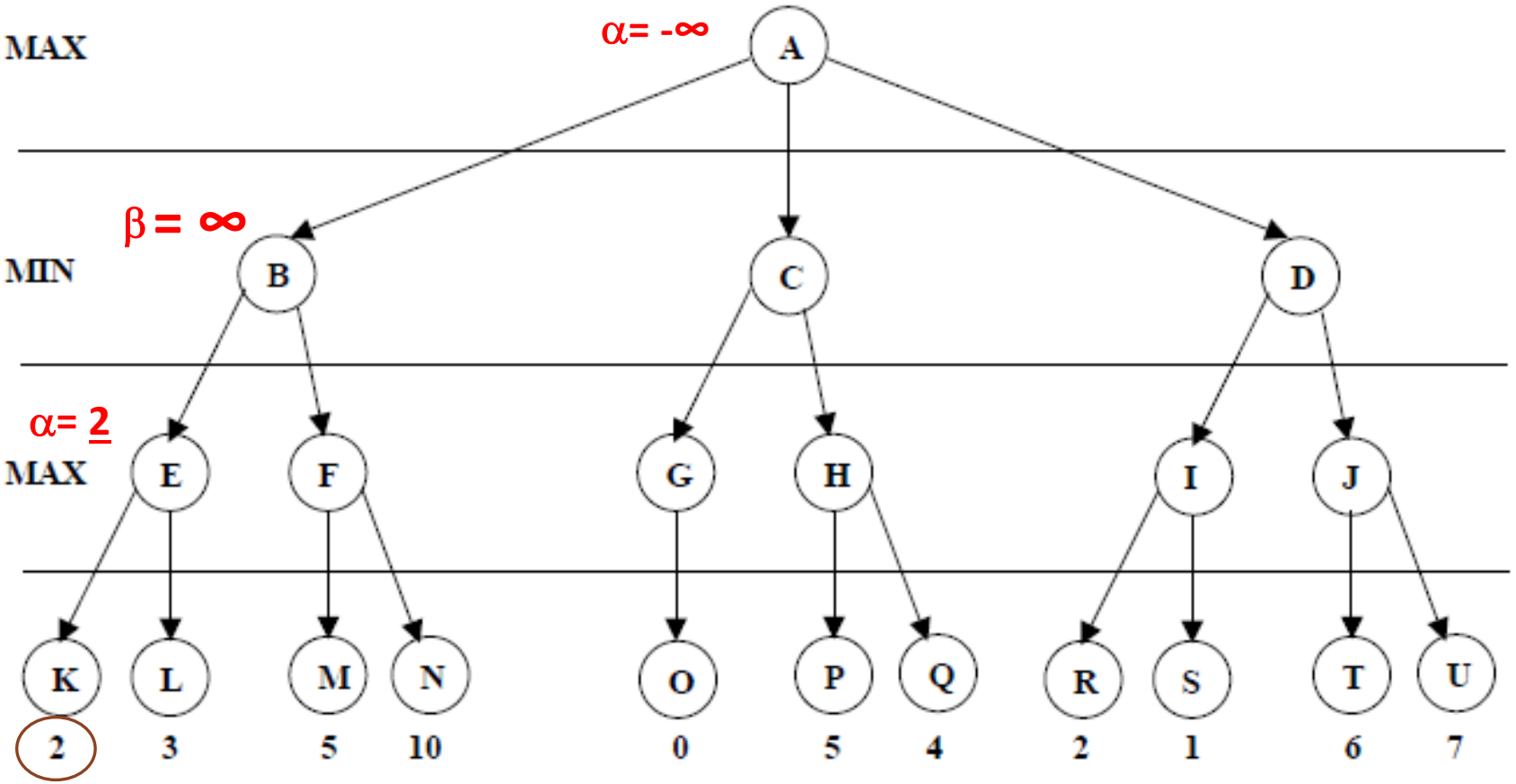




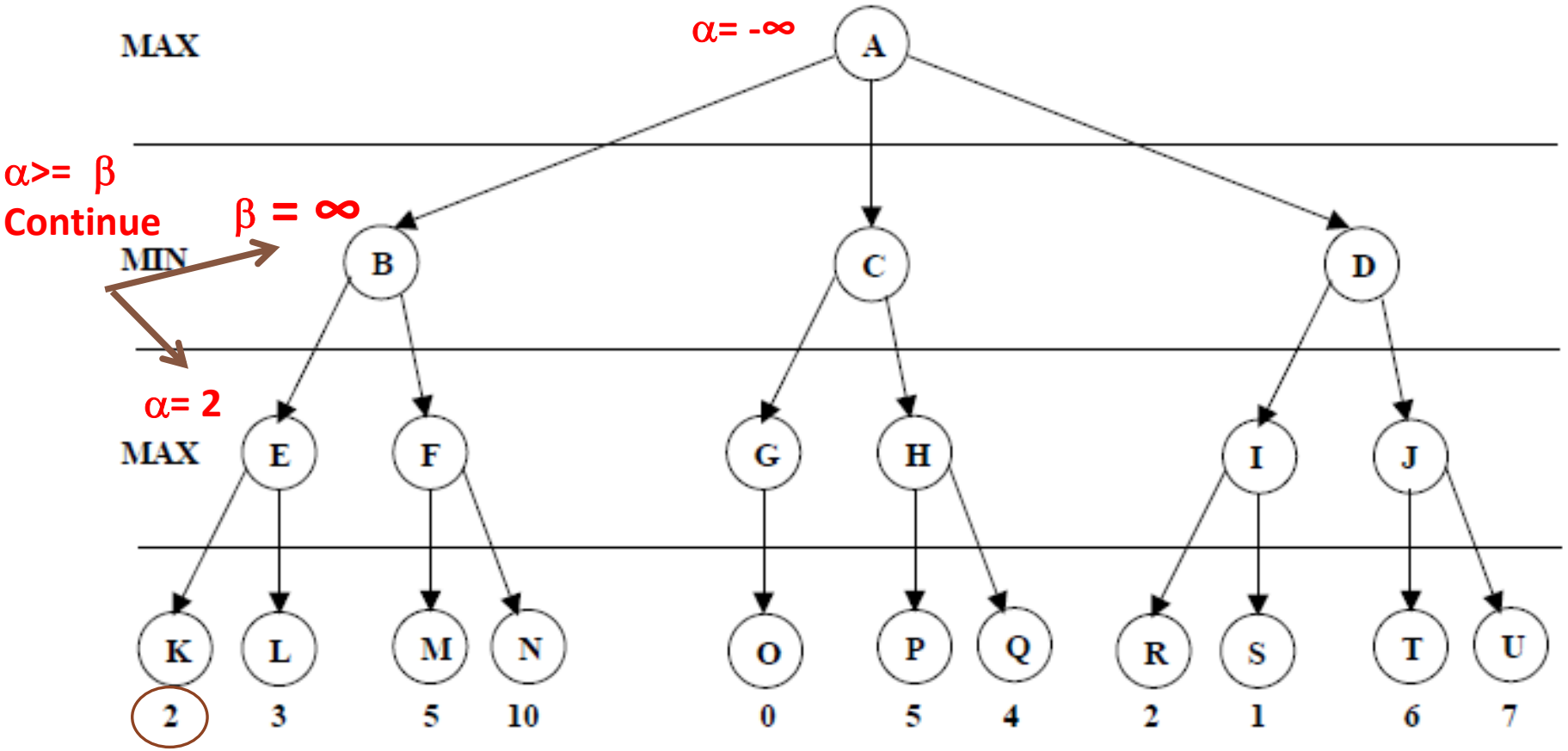
β –α Pruning Example



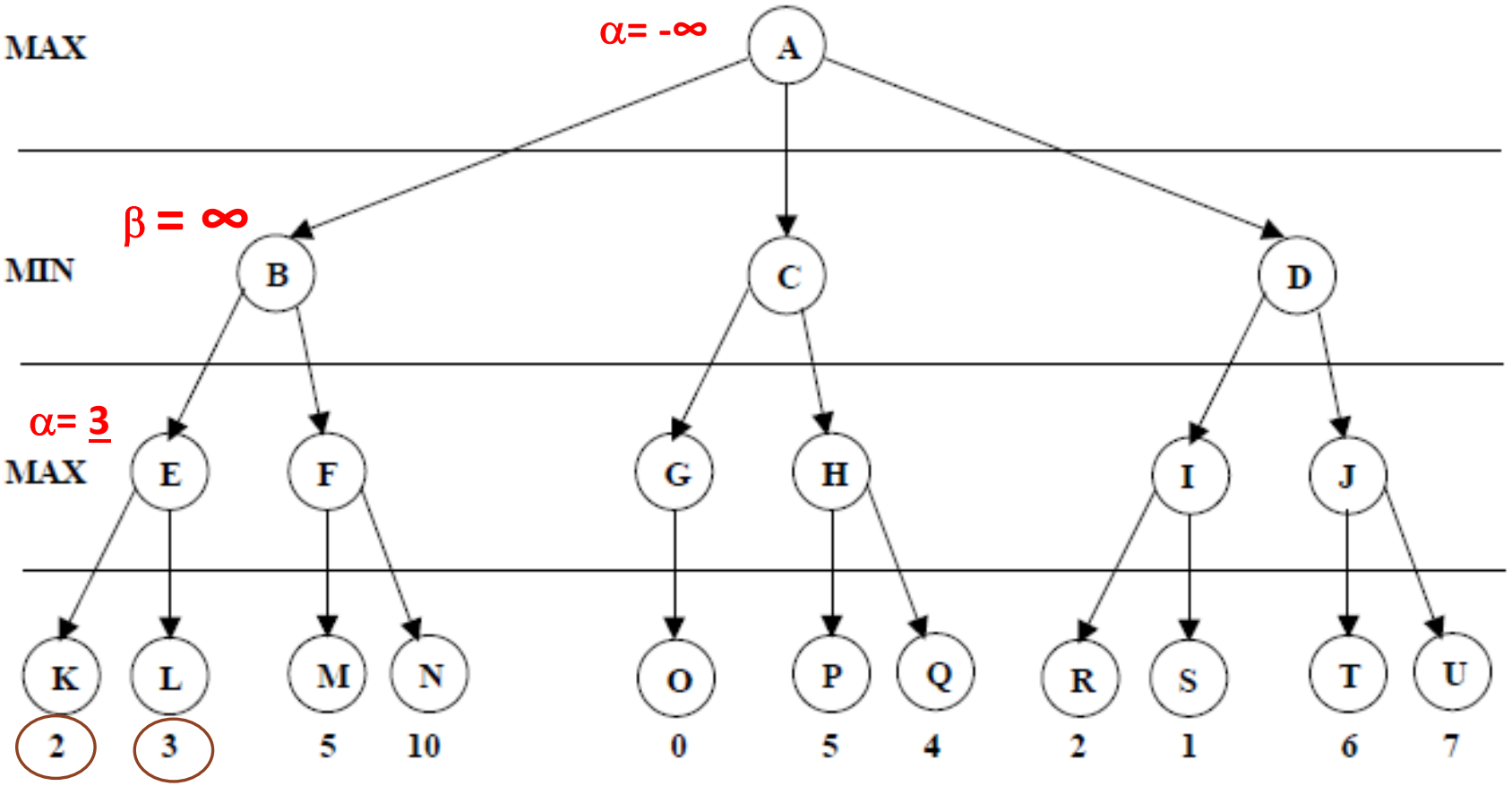
β –α Pruning Example



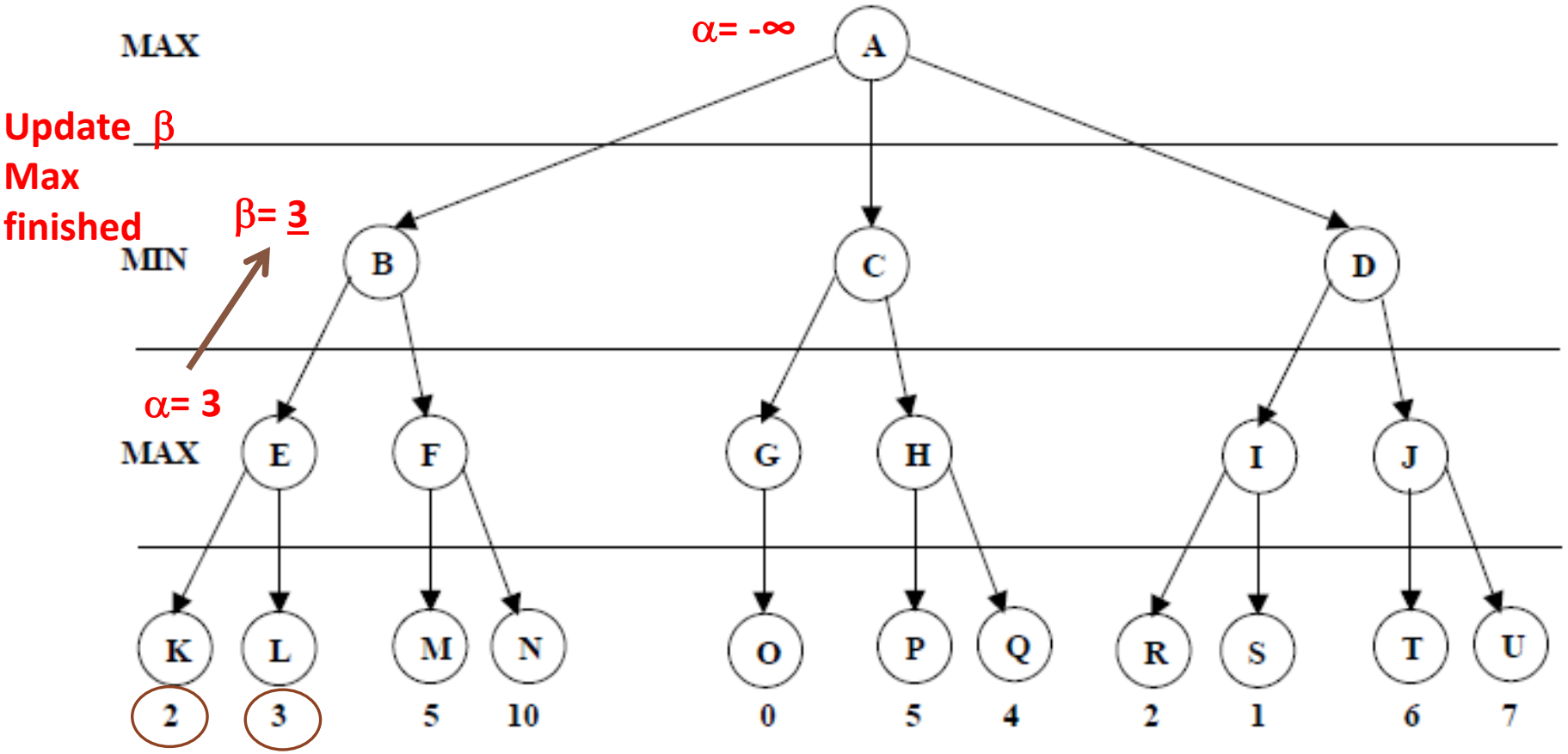
β –α Pruning Example



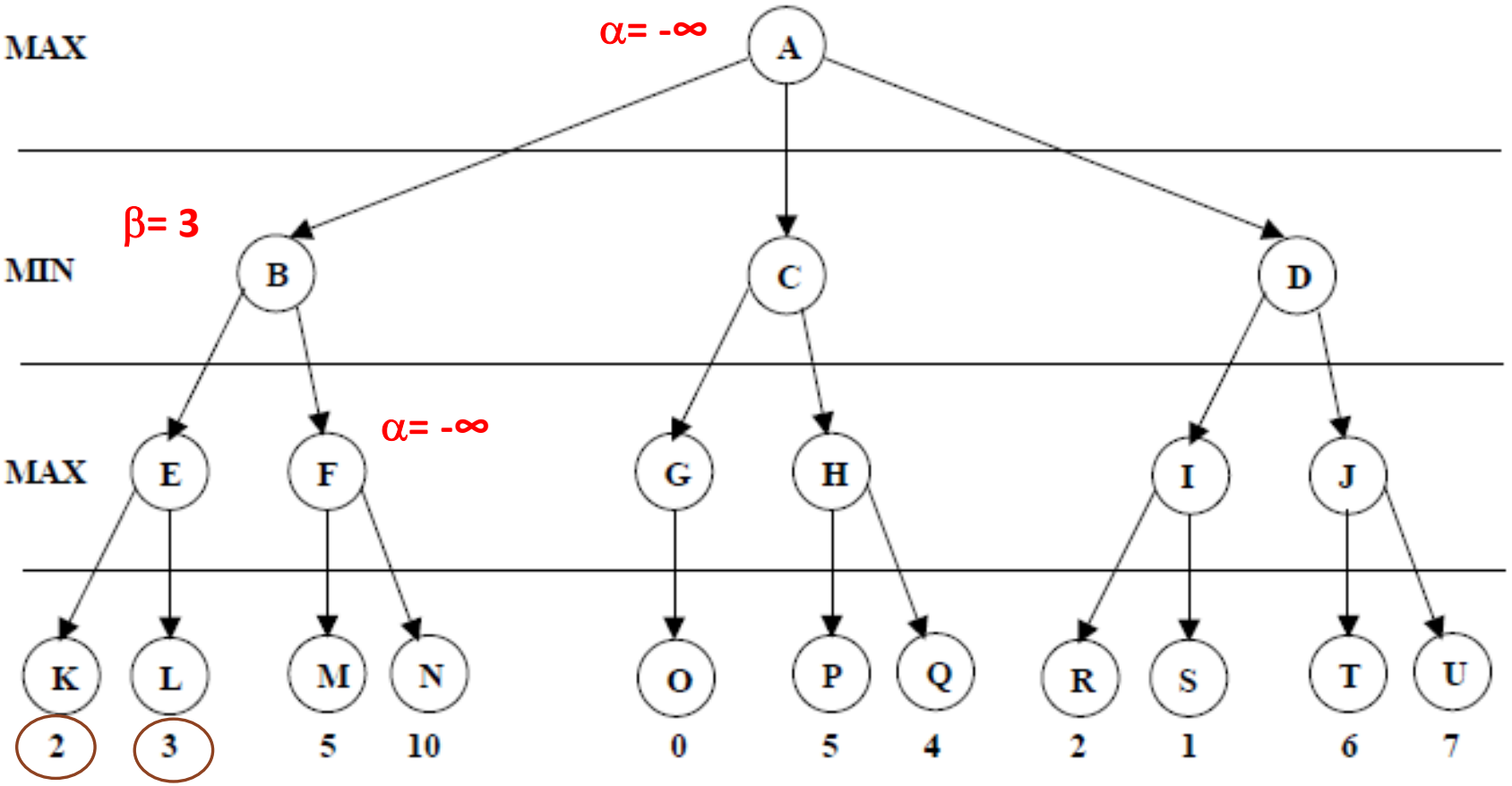
β –α Pruning Example



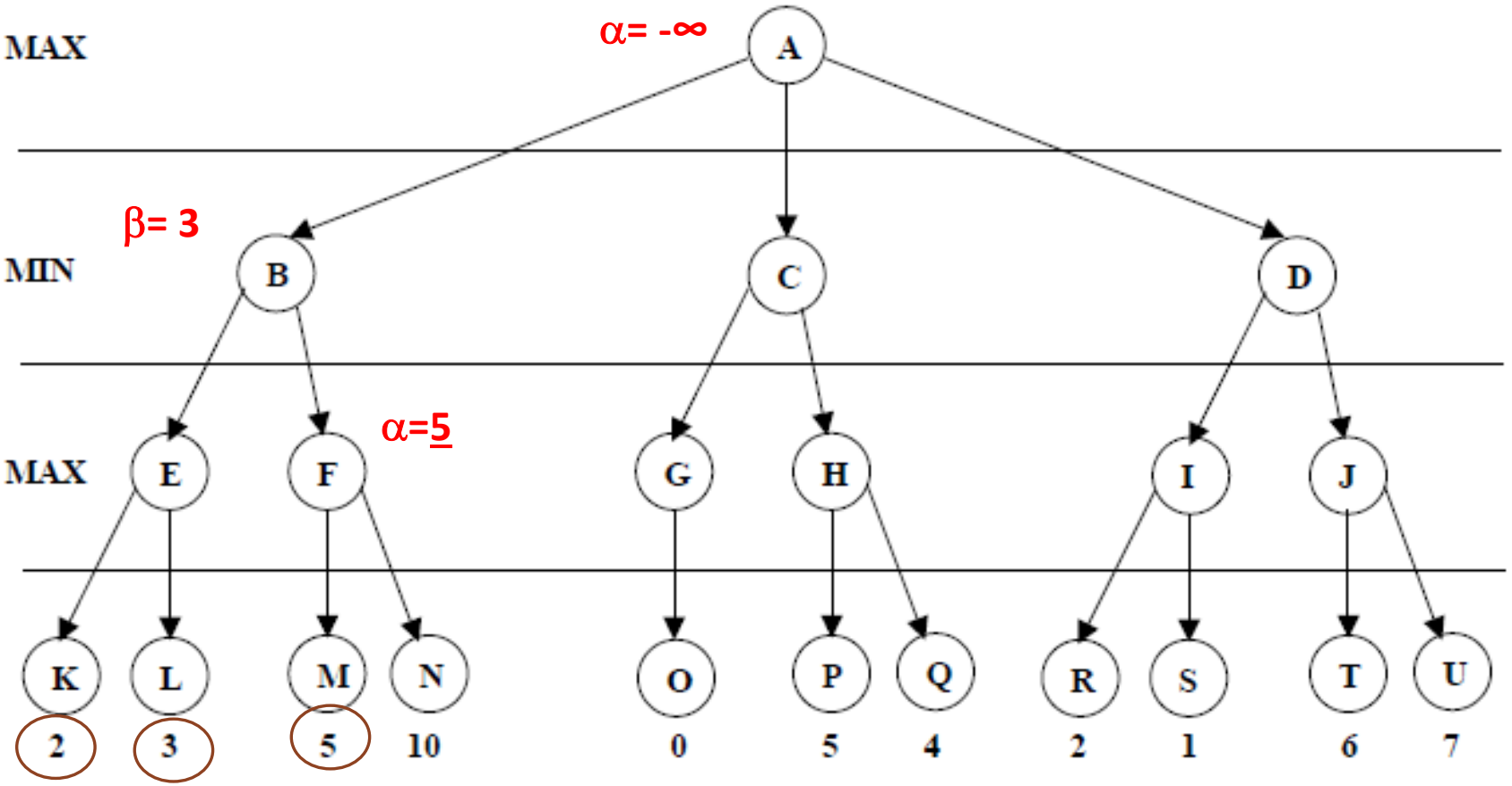
β –α Pruning Example



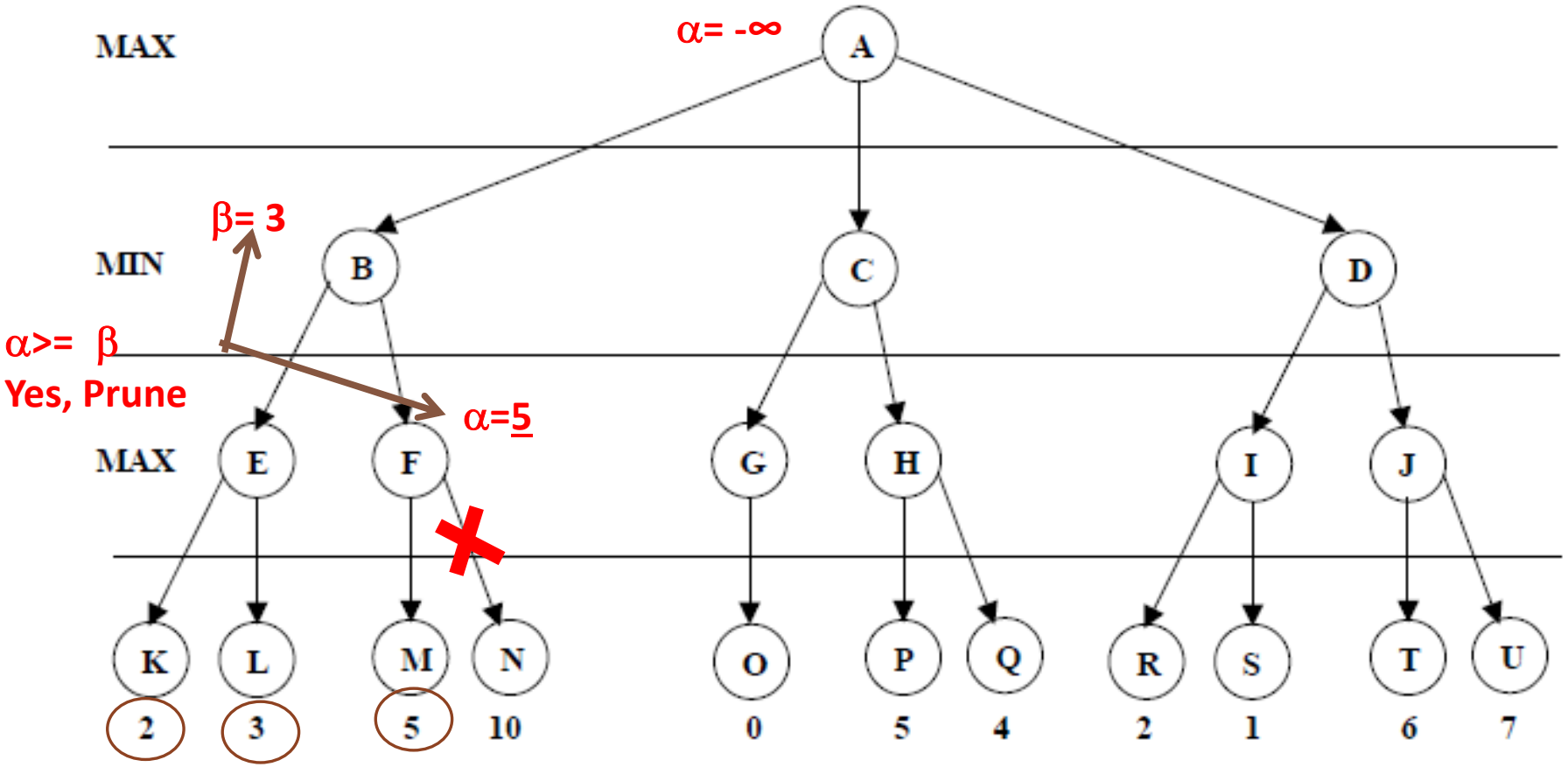
β –α Pruning Example



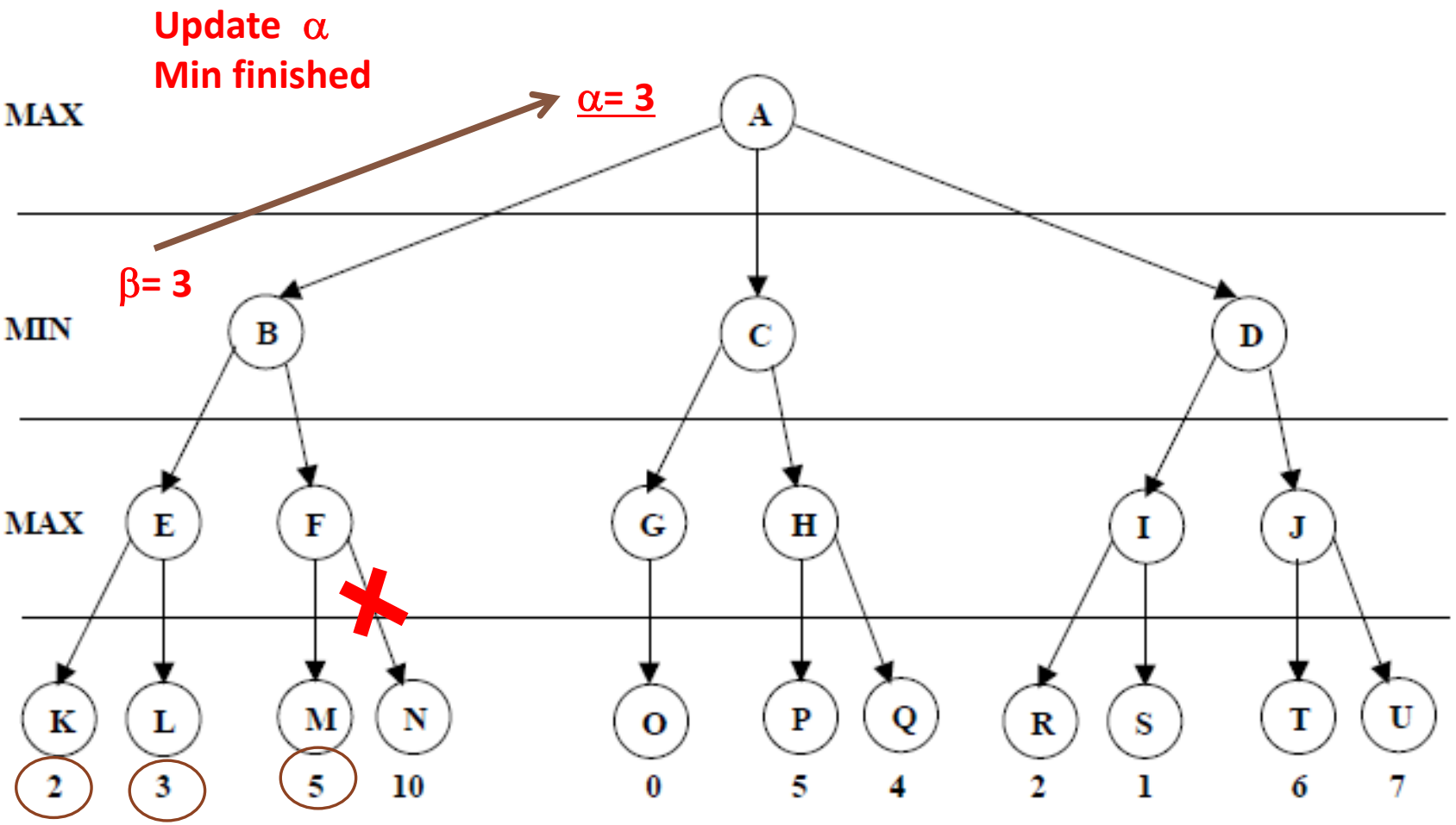
β –α Pruning Example



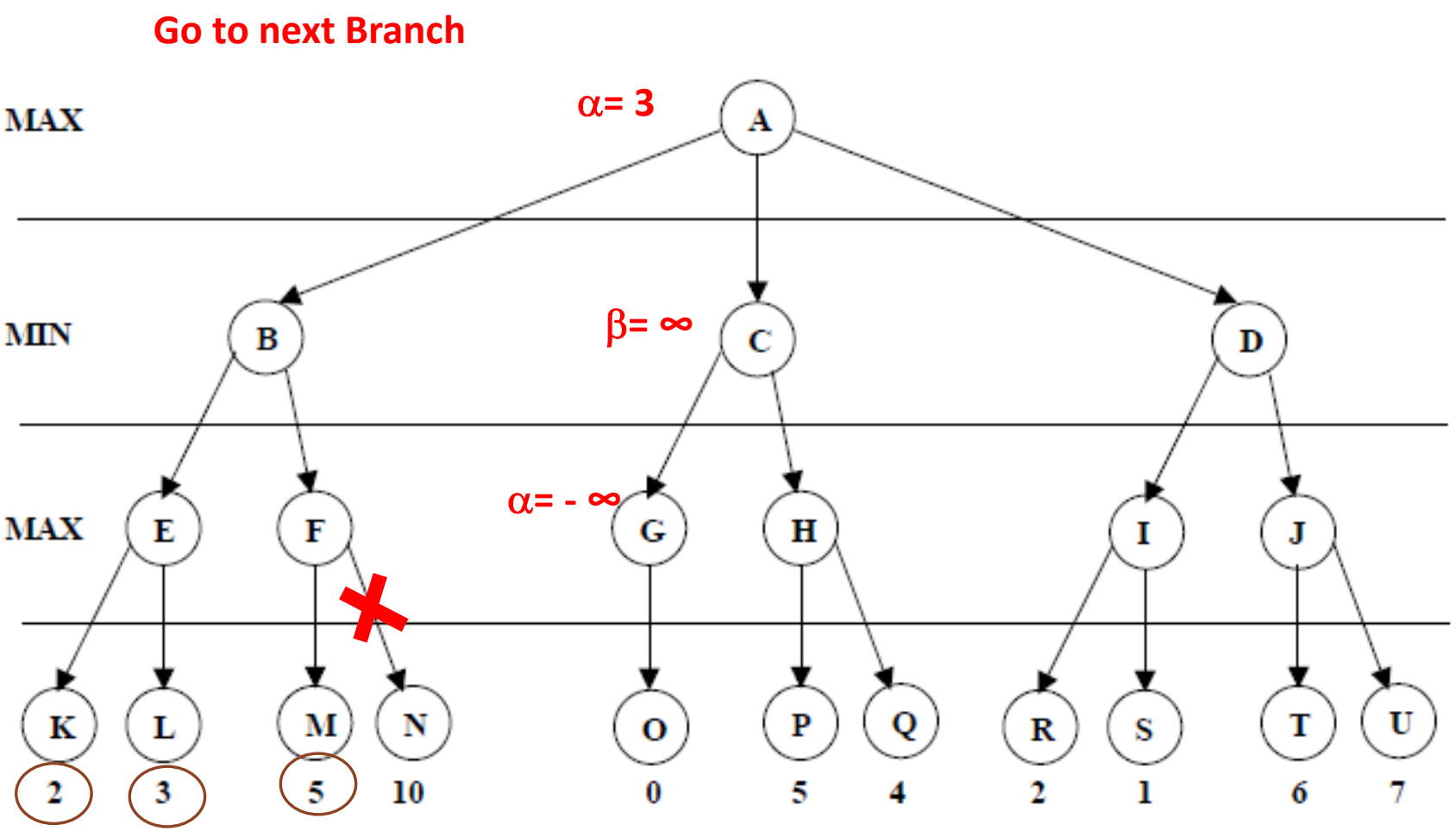
β –α Pruning Example



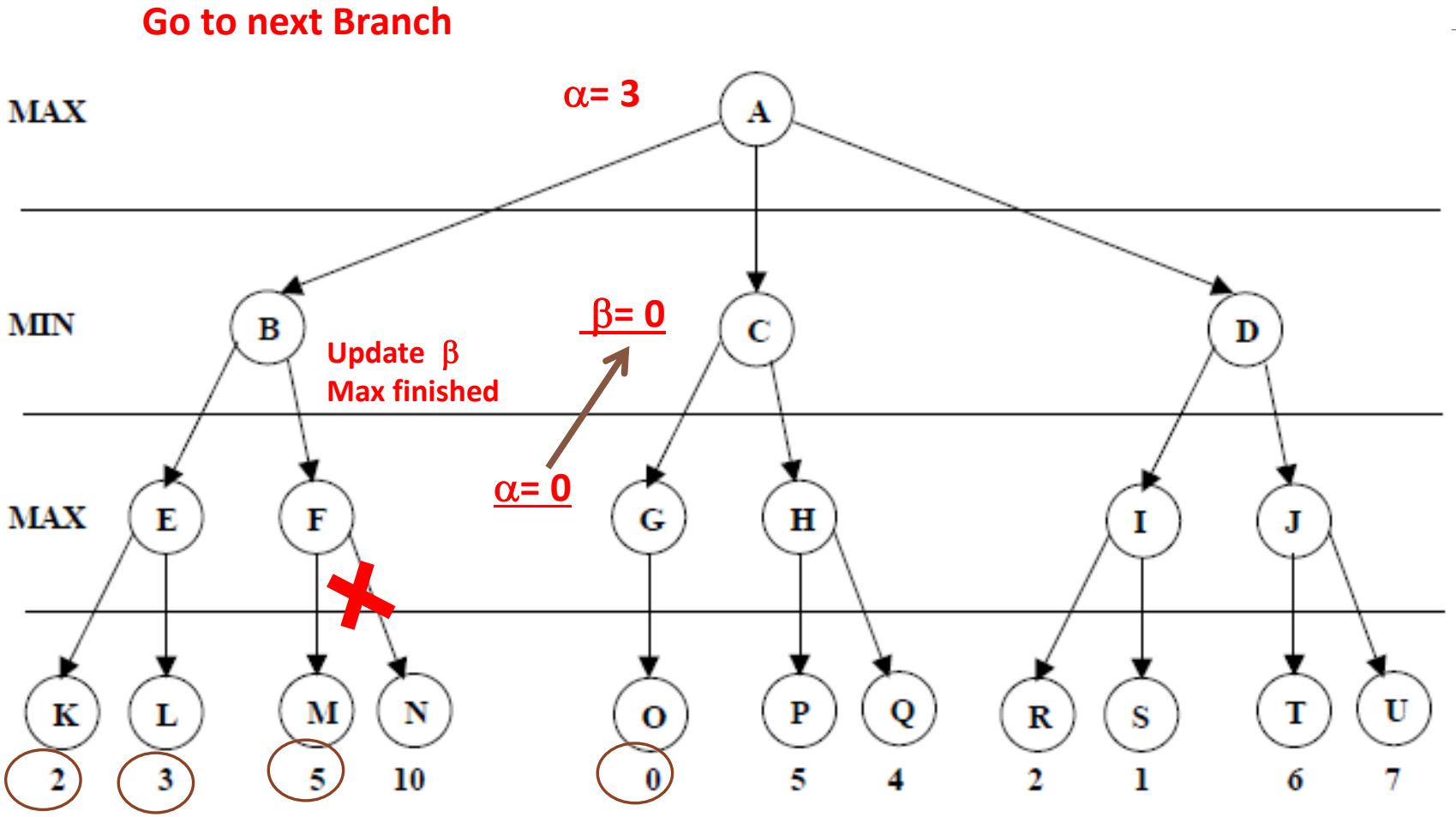
β –α Pruning Example



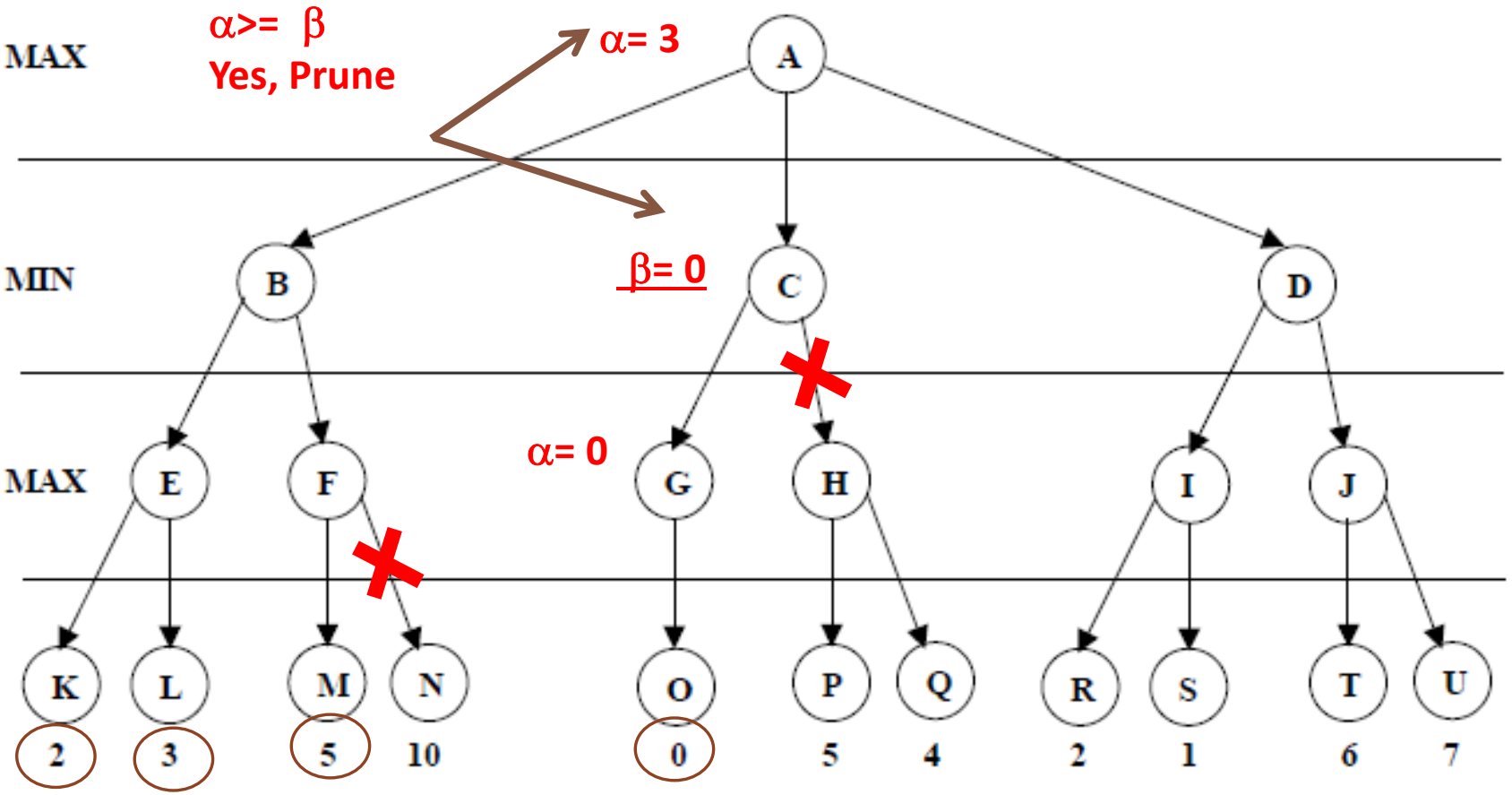
β –α Pruning Example



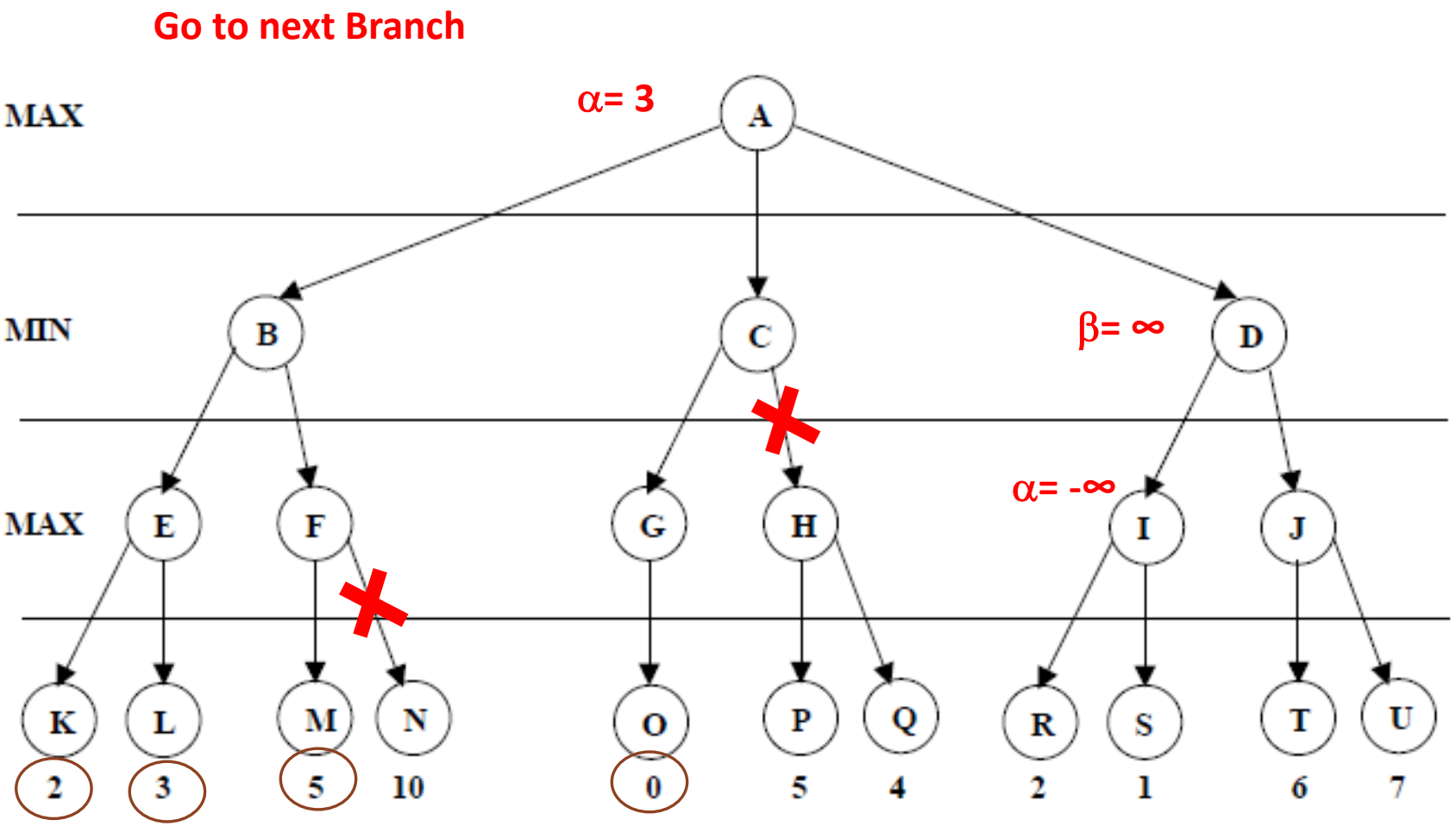
β –α Pruning Example



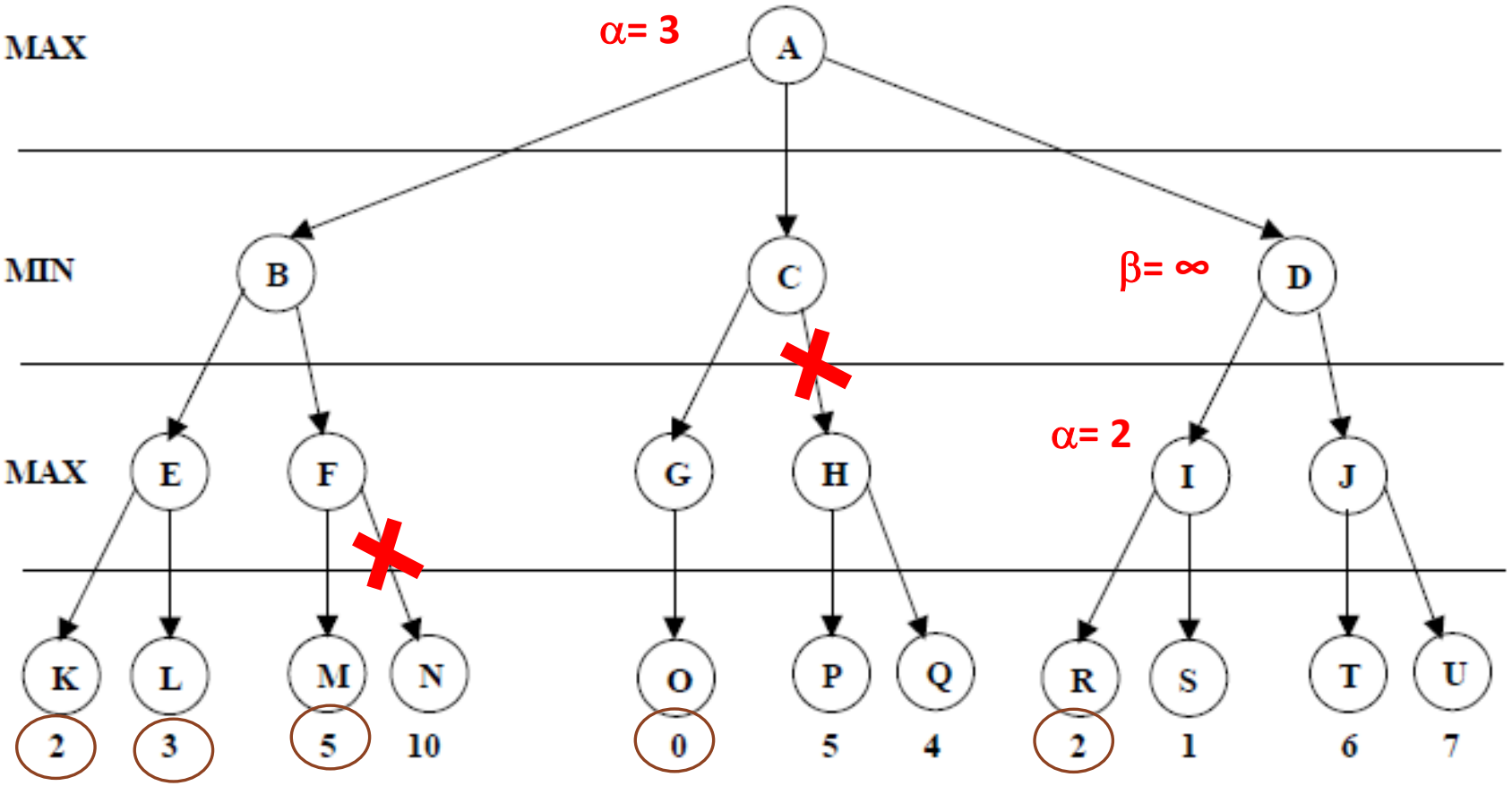
β –α Pruning Example



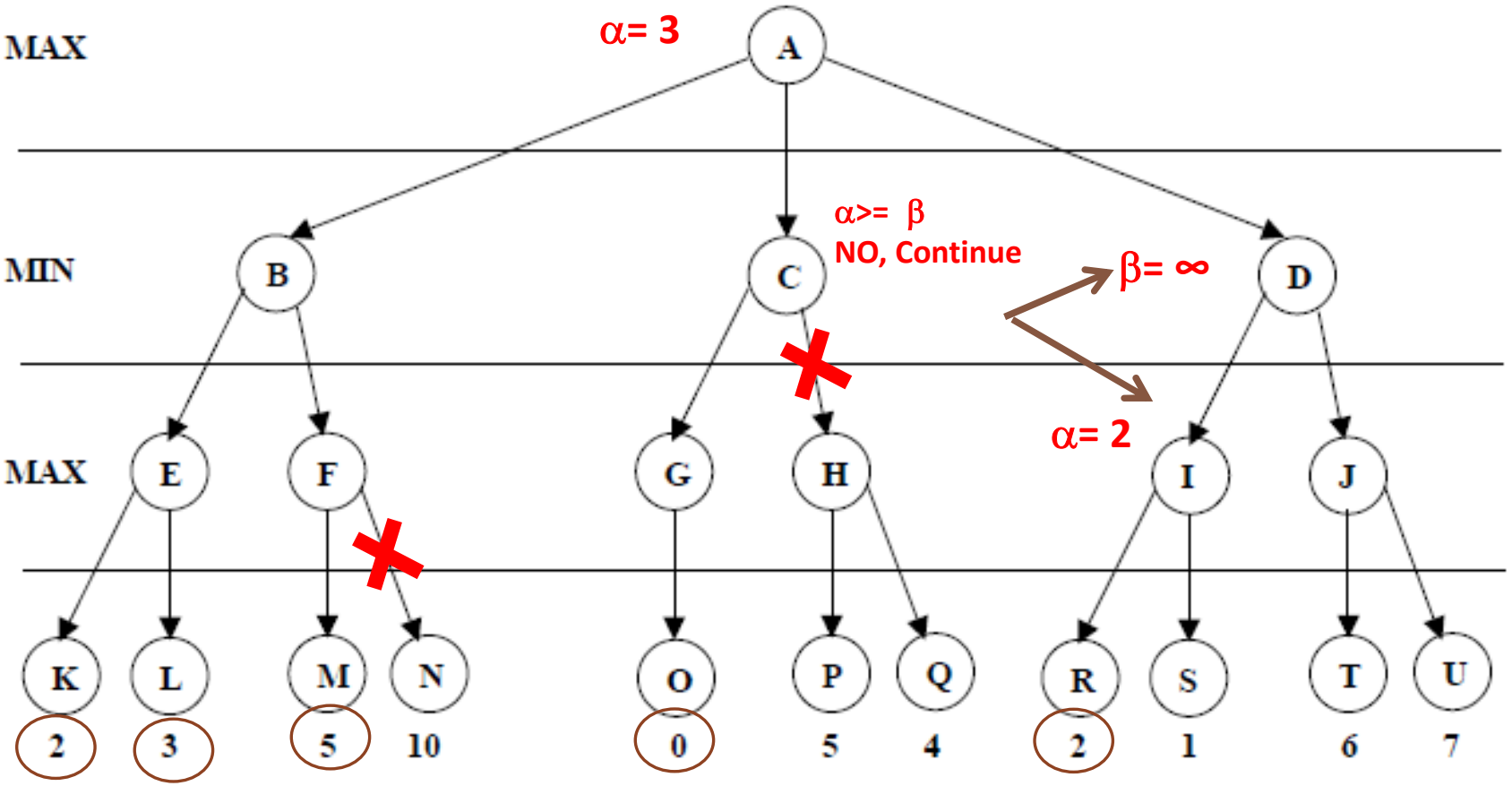
β –α Pruning Example



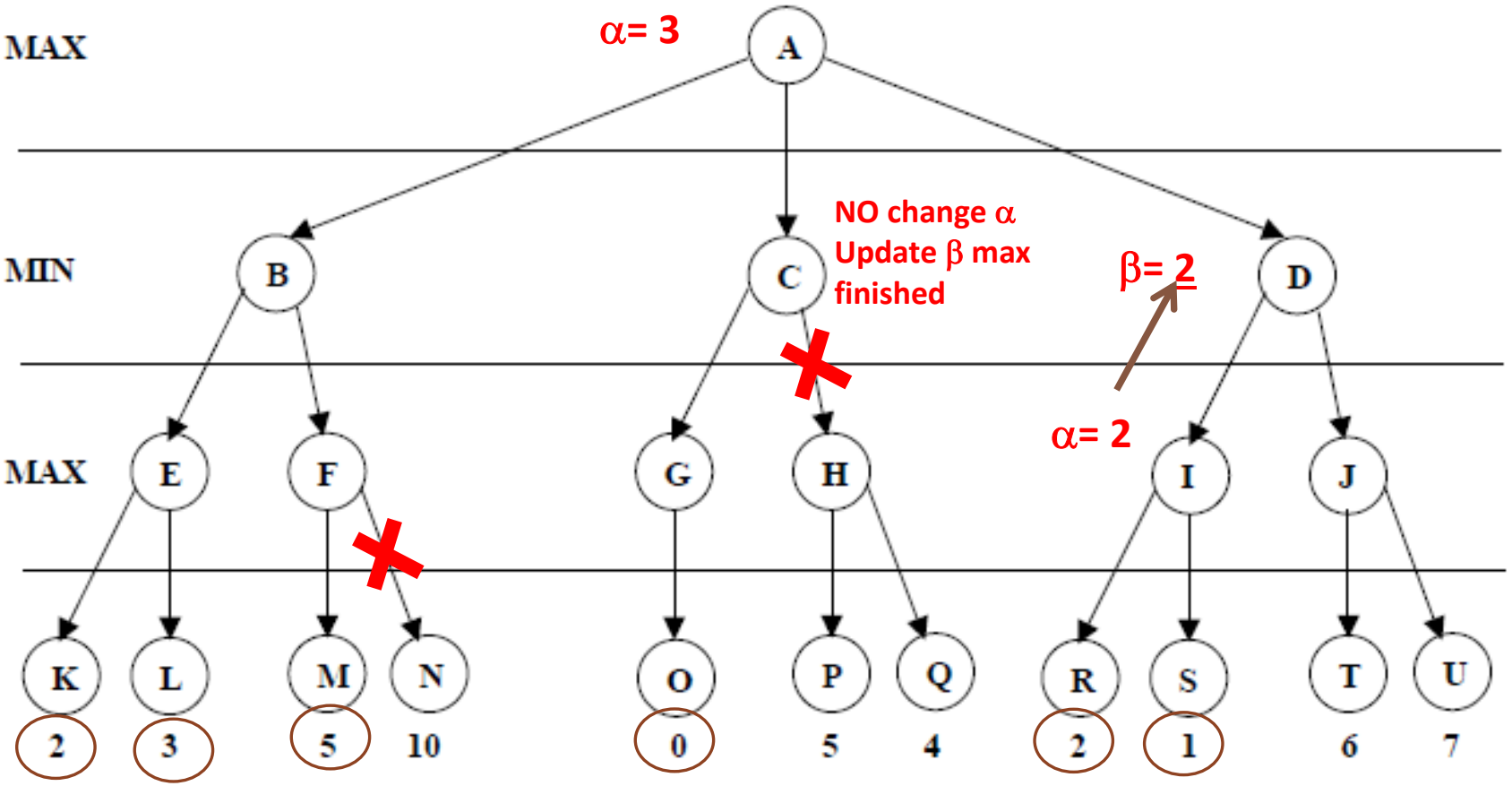
β –α Pruning Example



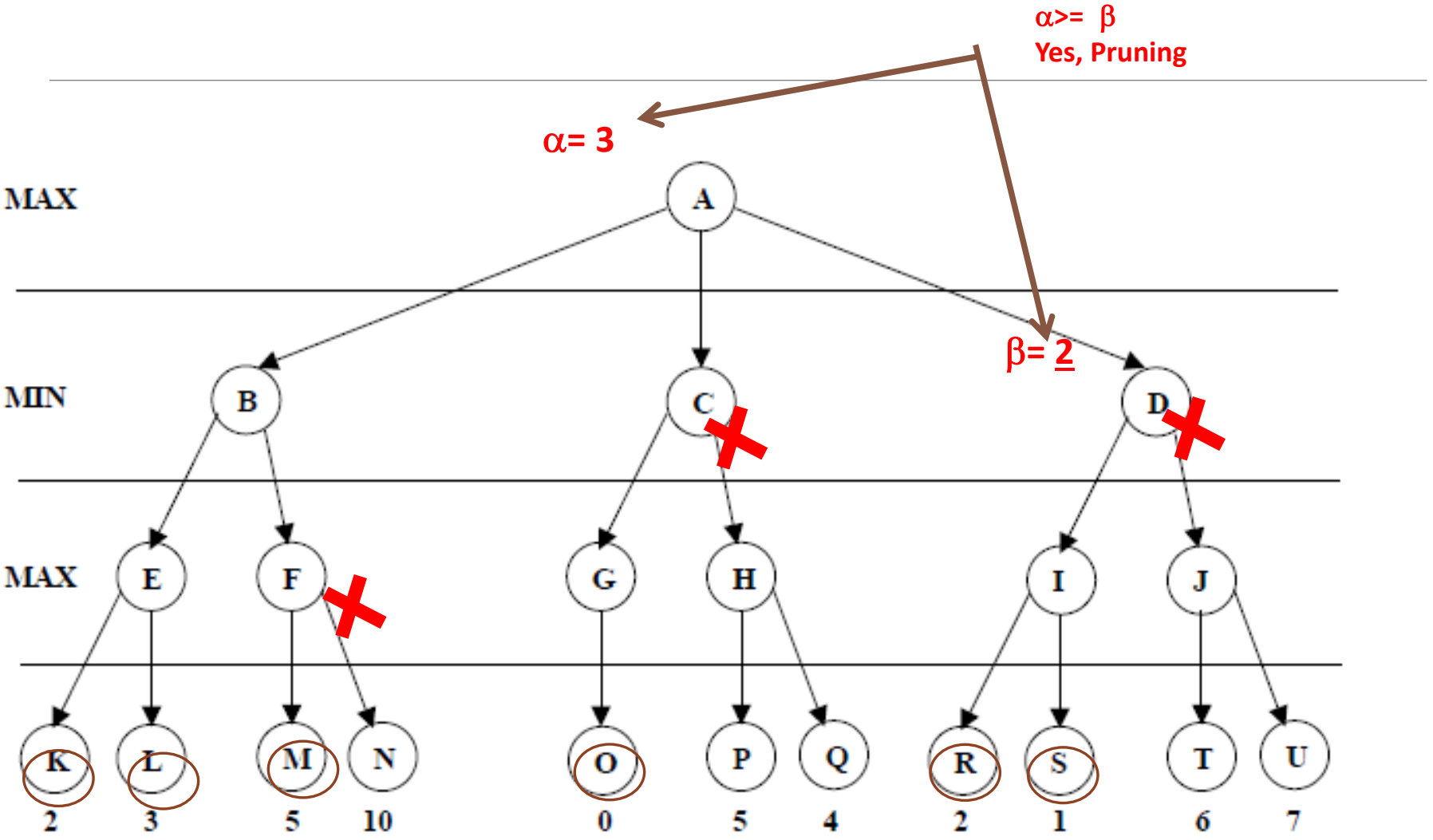
β –α Pruning Example



β –α Pruning Example



β –α Pruning Example



THANKS